

Workflow Orchestration

Workflow Orchestration Layer

Overview

This document defines the **Workflow Orchestration Layer** for your event-driven, Redis-backed, polyglot agent system. It is the component that *actually runs workflows* by coordinating:

- Workflow definitions
- Workflow execution state
- Node execution triggers
- Edge propagation
- Pause/resume logic
- Multi-agent pipelines

This layer sits between the **Workflow Engine**, **Gateway**, **Agent Harness**, and **MongoDB Persistence Layer**, ensuring that workflows execute deterministically and can be paused, resumed, or recovered.

Folder Structure

```
/workflow
/orchestration
  orchestrator.ts
  nodeRunner.ts
  edgeRunner.ts
  resumeManager.ts
  stateUpdater.ts
```

 File: orchestration/orchestrator.ts

```

import { WorkflowDefinitionModel } from "../../db/models/WorkflowDefinition";
import { WorkflowExecutionModel } from "../../db/models/WorkflowExecution";
import { WorkflowNodeStateModel } from "../../db/models/WorkflowNodeState";
import { runNode } from "./nodeRunner";
import { propagateEdge } from "./edgeRunner";
import { updateExecutionState } from "./stateUpdater";

export class WorkflowOrchestrator {
  async startExecution(workflowId: string) {
    const definition = await WorkflowDefinitionModel.findOne({ workflowId });
    if (!definition) throw new Error("Workflow not found");

    const execution = await WorkflowExecutionModel.create({
      workflowId,
      executionId: crypto.randomUUID(),
      state: "running"
    });

    await updateExecutionState(execution.executionId, "running");

    // Start with nodes that have no inputs
    const startNodes = definition.nodes.filter(n => n.inputs.length === 0);

    for (const node of startNodes) {
      await runNode(execution.executionId, node);
    }

    return execution.executionId;
  }

  async handleNodeCompletion(executionId: string, nodeId: string, output: any) {
    const nodeState = await WorkflowNodeStateModel.findOne({ executionId, nodeId });
    if (!nodeState) return;

    nodeState.state = "completed";
    nodeState.lastEvent = output;
    await nodeState.save();

    const definition = await WorkflowDefinitionModel.findOne({ workflowId:
nodeState.workflowId });
    const edges = definition.edges.filter(e => e.fromNodeId === nodeId);

    for (const edge of edges) {
      await propagateEdge(executionId, edge, output);
    }
  }
}

```

File: orchestration/nodeRunner.ts

```
import { WorkflowNodeStateModel } from "../../db/models/WorkflowNodeState";
import { redisPub } from "../../gateway/redis/pub";

export async function runNode(executionId: string, node: any) {
  await WorkflowNodeStateModel.create({
    executionId,
    nodeId: node.id,
    state: "running"
  });

  // Trigger agent execution via Redis
  await redisPub.publish(
    "tasks",
    JSON.stringify({
      type: "task.created",
      taskId: node.id,
      sessionId: node.sessionId
    })
  );
}
```

File: orchestration/edgeRunner.ts

```
import { WorkflowEdgeStateModel } from "../../db/models/WorkflowEdgeState";
import { runNode } from "../nodeRunner";

export async function propagateEdge(executionId: string, edge: any, value: any) {
  await WorkflowEdgeStateModel.create({
    executionId,
    edgeId: edge.id,
    lastValue: value,
    operator: edge.operator
  });

  // Trigger downstream node
  await runNode(executionId, { id: edge.toNodeId, sessionId: edge.sessionId });
}
```

File: orchestration/resumeManager.ts

```
import { WorkflowNodeStateModel } from "../../db/models/WorkflowNodeState";
import { redisPub } from "../../gateway/redis/pub";

export async function resumeNode(executionId: string, nodeId: string, choice: string) {
  const nodeState = await WorkflowNodeStateModel.findOne({ executionId, nodeId });
  if (!nodeState) return;

  nodeState.state = "running";
  await nodeState.save();

  await redisPub.publish(
    "tasks",
    JSON.stringify({
      type: "task.resume",
      taskId: nodeId,
      choice
    })
  );
}
```

File: orchestration/stateUpdater.ts

```
import { WorkflowExecutionModel } from "../../db/models/WorkflowExecution";

export async function updateExecutionState(executionId: string, newState: string) {
  const execution = await WorkflowExecutionModel.findOne({ executionId });
  if (!execution) return;

  execution.state = newState;
  execution.updatedAt = new Date();
  await execution.save();
}
```

Integration Notes

Gateway Responsibilities

- Start workflow execution
- Forward agent events to orchestrator

- Trigger resume actions

Agent Harness Responsibilities

- Execute nodes
- Emit completion events
- Emit human-in-the-loop events

Workflow Engine Responsibilities

- Subscribe to session events
- Trigger orchestrator callbacks

MongoDB Responsibilities

- Store workflow execution state
- Store node/edge state
- Store checkpoints

Summary

This Workflow Orchestration Layer provides:

- Deterministic workflow execution
- Node and edge propagation
- Pause/resume logic
- Multi-agent pipeline support
- Full integration with Redis, Gateway, Agent Harness, and MongoDB

It completes the workflow subsystem for your MVP and ensures future scalability.